

FATEC POMPEIA — SHUNJI NISHIMURA

Introdução à Computação em Nuvem · ISW055 · 161_SIST. INTELIGENTES_N

P1

Relatório bimestral de atividades

Avaliação individual de prestação de contas · 2026.2

Felipe Takayuki Tanaka Takahashi

Sumário

1	Introdução	2
2	Metodologia	2
3	Quadro de entregas	3
4	Atividades realizadas	4
4.1	Atividade 1 — Agenda telefônica em Flask	4
4.2	Atividade 2 — Catálogo de filmes — Tom Hanks	8
4.3	Atividade 3 — Desacoplando o login — microsserviço de autenticação	10
4.4	Atividade 4 — Controle de acesso por papel — RBAC	13
4.5	Atividade 5 — Logs e auditoria	16
4.6	Atividade 6 — Upload e perfil de usuário	18
5	Atividades extras (opcional)	20
5.1	Atividade E3 — Observabilidade — health checks e métricas	20
6	Considerações finais	22
7	Declaração de autoria	22

1 Introdução

A disciplina de Introdução à Computação em Nuvem (ISW055), ministrada pelo Prof. Allan Lincoln Rodrigues Siriani na Faculdade de Tecnologia de Pompeia (Fatec), tem como objetivo central capacitar os estudantes nos fundamentos práticos da engenharia de software distribuída, padrões de microsserviços, isolamento por containerização e infraestrutura escalável em nuvem. Ao longo do primeiro bimestre do semestre letivo de 2026.2, a disciplina adotou uma metodologia de aprendizado baseada na evolução incremental e contínua de um ecossistema real de aplicações, partindo de um monólito inicial de nivelamento em sala até a orquestração de uma plataforma moderna baseada em múltiplos serviços independentes.

O projeto fio condutor do bimestre consistiu no desenvolvimento de um sistema completo de Catálogo de Filmes com foco na filmografia do ator Tom Hanks, alimentado de forma dinâmica pela API pública do The Movie Database (TMDB). Esse sistema evoluiu iterativamente através de entregas semanais planejadas, incorporando de ponta a ponta o desacoplamento de serviços, autenticação centralizada com JSON Web Tokens (JWT), fluxo seguro de recuperação de senha por e-mail transacional, autorização baseada em papéis (RBAC com enforcement HTTP 403), observabilidade com microsserviço de auditoria sobre Redis Streams e armazenamento de arquivos de mídia (avatars de perfil) em Object Storage compatível com AWS S3 (MinIO).

Este relatório consolida e presta contas de todas as atividades desenvolvidas e entregues individualmente pelo aluno durante o período avaliado. O documento está organizado em seções rigorosas que apresentam: a metodologia de desenvolvimento e rastreamento; o quadro sinóptico de entregas confrontando as datas planejadas e as datas efetivamente realizadas; as fichas detalhadas de cada atividade com a fundamentação técnica das decisões arquiteturais e suas respectivas evidências visuais (histórico de commits no Git, vínculos com as issues oficiais no GitHub e registros do sistema em execução); as atividades extras de observabilidade; e as considerações finais sobre as lições aprendidas.

2 Metodologia

O ciclo de desenvolvimento de todas as atividades foi pautado pelas melhores práticas da indústria de software em termos de versionamento semântico, rastreabilidade bidirecional e isolamento de dependências. Com exceção do nivelamento inicial da Atividade 1 (desenvolvido presencialmente em sala de aula com repositório dedicado Felipe-Takayuki/lista-de-produtos), todas as entregas do projeto do catálogo de filmes foram construídas de forma estritamente cumulativa no mesmo repositório público do GitHub (Felipe-Takayuki/atividade-filmes). Essa abordagem garantiu a integridade histórica da arquitetura, demonstrando como cada microsserviço foi introduzido sem quebrar os requisitos funcionais estabelecidos anteriormente.

O ambiente de desenvolvimento e testes foi executado em ambiente Linux (distribuição Fedora), adotando o Docker e o Docker Compose como base padronizada de containerização. A infraestrutura foi configurada com isolamento rigoroso de redes virtuais (app-network em modo bridge): apenas a porta pública :3000 do serviço de catálogo foi exposta para o host, enquanto os demais componentes (auth-service, log-service, redis, mariadb e minio) operaram estritamente em rede interna via diretivas expose, prevenindo a exposição indevida de superfícies de ataque e conflitos de portas em ambientes compartilhados como o Portainer.

Para atender aos critérios de transparência e auditoria exigidos pelo professor, a coleta das datas e horas das entregas baseou-se estritamente em duas fontes verificáveis: o histórico criptográfico de commits do Git e o registro público de Issues no GitHub. Os carimbos de data e hora foram conferidos através do comando oficial `git log -1 --date=format:"%d/%m/%Y %H:%M" --format="%h %ad"` executado no terminal local e cruzados com as Issues abertas no repositório (issues #1, #2, #3, #4 e #5). Para cada atividade, foram consolidados dois tipos de evidências: o print do terminal demonstrando o hash do commit, o autor, a data/hora e o repositório remoto; e o print do sistema em execução, comprovando o correto funcionamento das regras de negócio, telas da interface e tratamentos de segurança.

3 Quadro de entregas

A tabela a seguir consolida o cronograma oficial de entregas do primeiro bimestre, confrontando as datas planejadas divulgadas no plano de ensino com as datas e horas reais de submissão comprovadas nas fichas da seção seguinte.

Nº	Atividade	Descrição	Data planejada	Data realizada	Situação
1	Agenda telefônica em Flask	Nivelamento em sala: sistema monolítico Flask + Jinja com persistência em JSON e migração para MySQL.	07/08/2026	07/08/2026 21:00	entregue
2	Catálogo de filmes — Tom Hanks	Consumo da API TMDb, persistência em MariaDB e segregação de favoritos e notas por usuário.	20/08/2026	20/08/2026 16:34	entregue
3	Desacoplando o login — microserviço de autenticação	Login, cadastro e esqueci-minha-senha em microserviço à parte na rede interna do Docker com tokens de 30 min.	28/08/2026	28/08/2026 13:56	entregue
4	Controle de acesso por papel — RBAC	Autorização baseada em papéis com enforcement no backend (403 para usuário comum), moderação e promoção.	04/09/2026	04/09/2026 16:50	entregue
5	Logs e auditoria	Novo microserviço log-service com Redis Streams (XADD/XREVRANGE) registrando logins, ações e tentativas 403.	25/09/2026	10/09/2026 15:48	entregue
6	Upload e perfil de usuário	Página de perfil com avatar no MinIO Object Storage (S3), metadados no MariaDB e proteção estrita RBAC 403.	02/10/2026	22/09/2026 15:38	entregue
E3	Observabilidade — health checks e métricas	Endpoints de saúde e integridade em cascata (/api/health) e dependência com service_healthy no Docker Compose.	sem prazo	10/09/2026 15:48	entregue

4 Atividades realizadas

4.1 Atividade 1 — Agenda telefônica em Flask

Descrição	Nivelamento em sala: sistema monolítico Flask + Jinja com persistência em JSON e migração para MySQL.
Data planejada	07/08/2026
Data realizada	07/08/2026 21:00
Situação	entregue
Evidência	Realizada em sala — telas do sistema web Flask rodando e relatórios da aplicação
Link	https://github.com/Felipe-Takayuki/lista-de-produtos

O que foi feito. Durante a aula presencial de nivelamento na Fatec, foi construída uma aplicação web monolítica utilizando a linguagem Python com o microframework Flask e motor de templates Jinja2. O sistema teve como propósito gerenciar uma lista de registros telefônicos/contatos, implementando as operações CRUD completas aderentes ao protocolo HTTP e padrões REST: POST para novo cadastro, GET para listagem de registros ativos, PUT para atualização de dados cadastrais e DELETE para envio de itens à lixeira lógica (**soft delete**). A arquitetura inicial baseou-se na persistência de dados em arquivo local JSON (`banco.json`) com campos cronológicos de auditoria (`created_at`, `updated_at`, `deleted_at`). Na sequência do aprendizado, a aplicação foi evoluída com container Docker, conexão relacional via PyMySQL, script DDL de criação de tabelas (`schema.sql`) e orquestração no Docker Compose com variáveis de ambiente configuradas para deploy no Portainer.

10/2/26, 12:51 PM

Lista Telefônica | Agenda de Contatos

Lista Telefônica

Agenda inteligente de contatos com discagem rápida, WhatsApp e backup seguro.

[Exportar vCard](#)[Exportar CSV](#)[Modo Escuro](#)

CONTATOS ATIVOS

4



CATEGORIAS ATIVAS

6



CONTATOS NA LIXEIRA

1

Novo Contato

Preencha os dados para adicionar à lista

Nome Completo *



Ex: Ana Clara Santos

Telefone / Celular *



(11) 98765-4321

E-mail (opcional)



exemplo@email.com

Categoria



Pessoal

Endereço (opcional)



Ex: Av. Paulista, 1000 - São Paulo,

Observações / Notas (opcional)



Ex: Ramal 204, aniversário em Mai

Salvar Contato

Contatos Ativos 4



Lixeira 1

localhost:5000/?tab=ativos

1/2

Figura 1: Lista Telefônica — cabeçalho, métricas de contatos e formulário de cadastro (página 1)

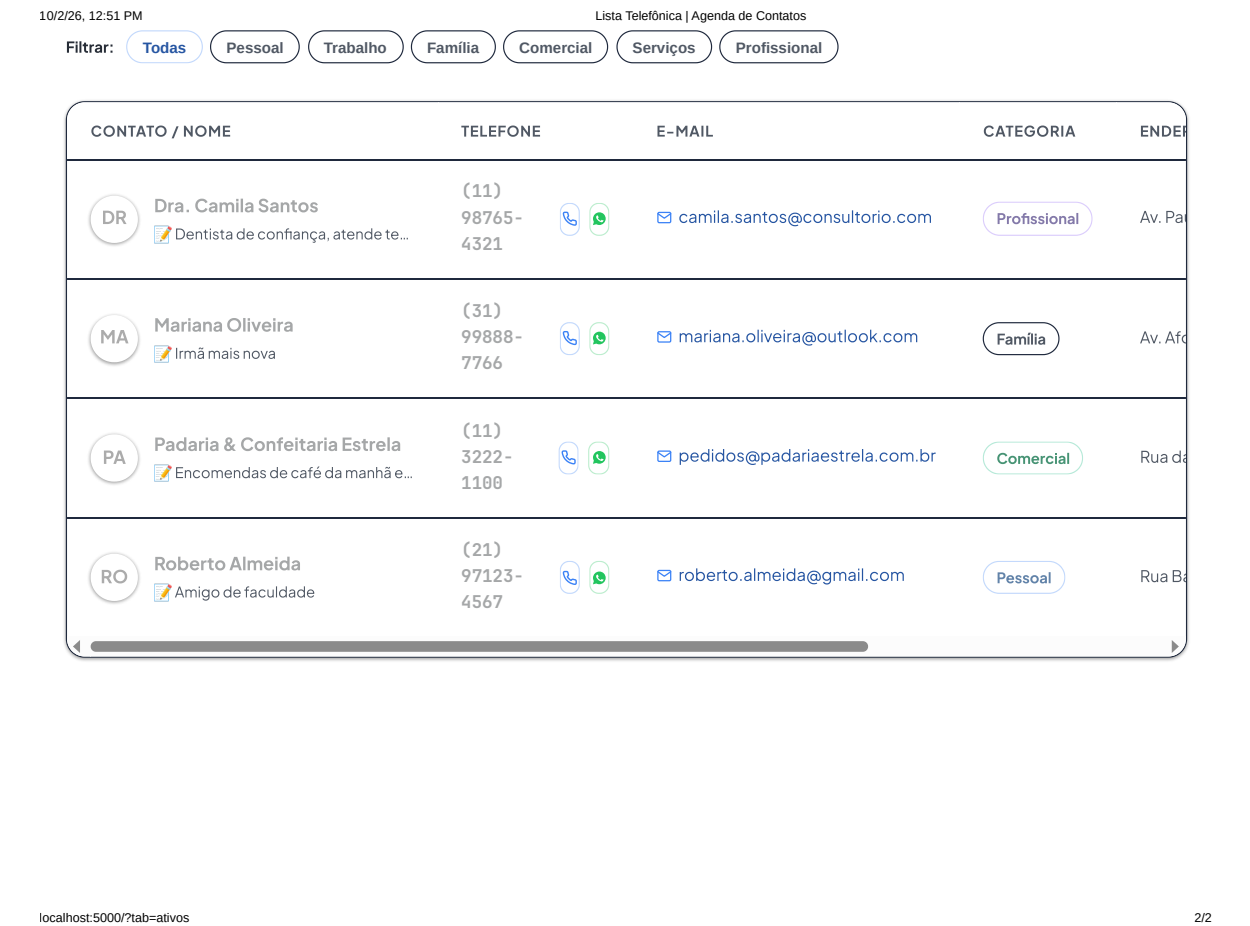


Figura 2: Lista Telefônica — listagem de contatos ativos e filtros por categoria (página 2)

10/2/26, 12:52 PM

Lista Telefônica | Agenda de Contatos

Filtrar: Todas Pessoal Trabalho Família Comercial Serviços Profissional

Contatos na Lixeira (1)

Contatos excluídos temporariamente. Você pode restaurá-los a qualquer momento ou excluí-los em definitivo.

Esvaziar Lixeira

CONTATO / NOME	TELEFONE	CATEGORIA	EXCLUÍDO EM	STATUS	AÇÕES
CA Carlos Eletricista ID #5	(19) 98111-2233	Serviços	2026-08-15 09:20:00	Apagado	Restaurar Excluir

localhost:5000/?tab=lixreira

2/2

Figura 3: Lista Telefônica — lixeira lógica de contatos com ações de restauração e exclusão definitiva (soft delete)

Dificuldades e como foram resolvidas. O principal desafio técnico consistiu em modelar a alternância entre itens ativos e itens na lixeira sem incorrer na perda permanente de dados durante a manipulação em JSON. A solução adotada foi a implementação do padrão de **soft delete**, marcando os registros com status "apagado" e registrando a data de exclusão, criando endpoints segregados (/dados para ativos e /lixreira para restauração/hard delete). Essa abordagem simplificou a migração posterior para as tabelas relacionais do MySQL.

4.2 Atividade 2 — Catálogo de filmes — Tom Hanks

Descrição	Consumo da API TMDb, persistência em MariaDB e segregação de favoritos e notas por usuário.
Data planejada	20/08/2026
Data realizada	20/08/2026 16:34
Situação	entregue
Evidência	GitHub — commit 230de48 + README + Issue #1 + print do catálogo
Link	https://github.com/Felipe-Takayuki/atividade-filmes/issues/1

O que foi feito. Nesta atividade inicial do projeto semestral, foi concebido o Catálogo de Filmes do ator Tom Hanks, integrando dinamicamente a API pública internacional do The Movie Database (TMDb). A aplicação foi estruturada com backend em Node.js/Express e persistência relacional no MariaDB 10.11. O sistema implementou autenticação segura através de JSON Web Tokens (JWT) trafegados em cookies HTTP-only, permitindo a segregação estrita dos dados: cada usuário cadastrado possui sua própria lista privada de filmes favoritos e pode publicar notas e comentários pessoais nos filmes listados. Foi desenvolvido o arquivo `docker-compose.yml` para orquestrar os serviços `catalogo` (porta pública :3000) e `mariadb` em rede interna privada. A entrega foi formalizada no repositório público com menção ao professor no README e abertura da Issue #1 (entrega da atividade).

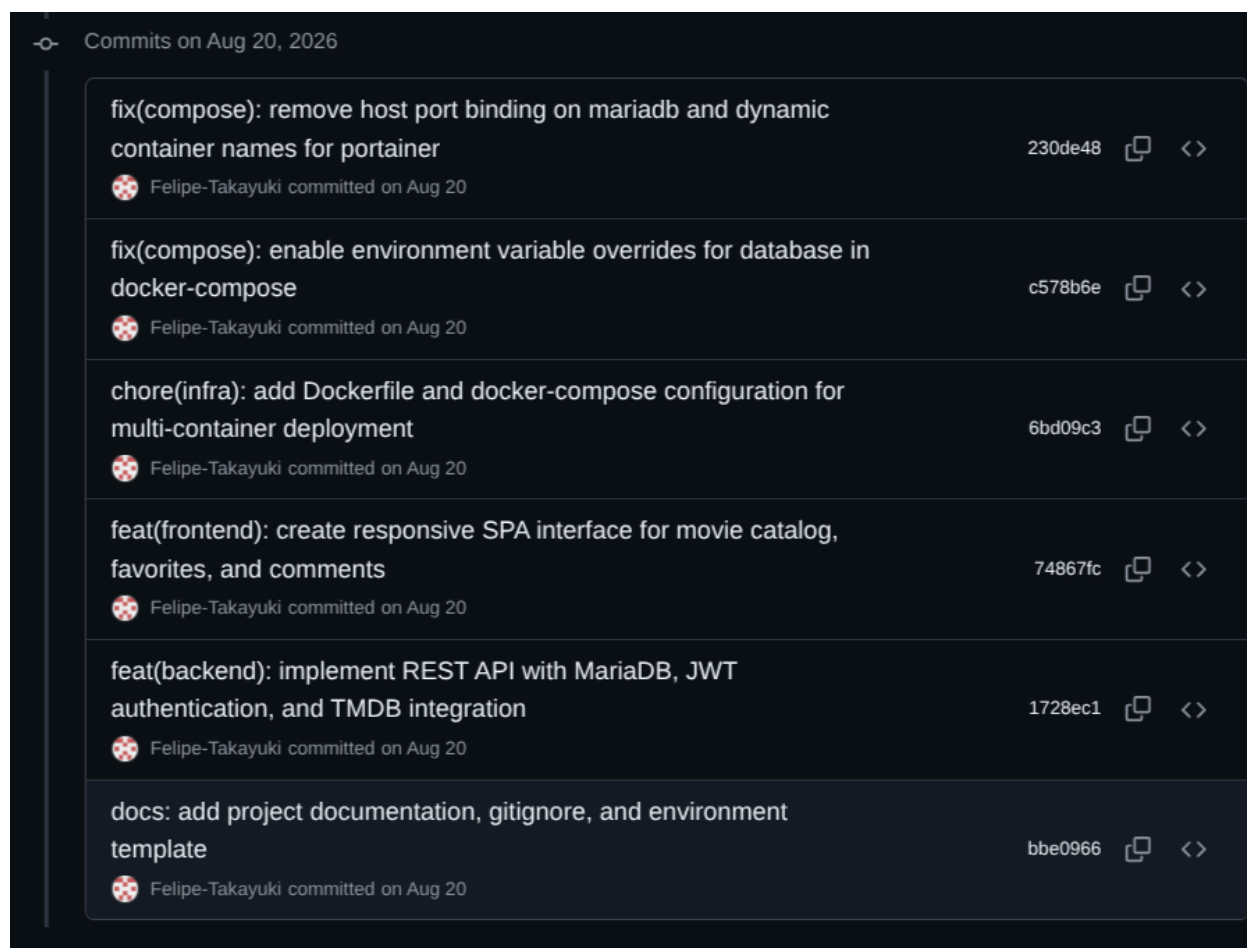


Figura 4: Atividade 2 — evidência da entrega (commit 230de48 e vínculo com a Issue 1 em 20/08/2026)

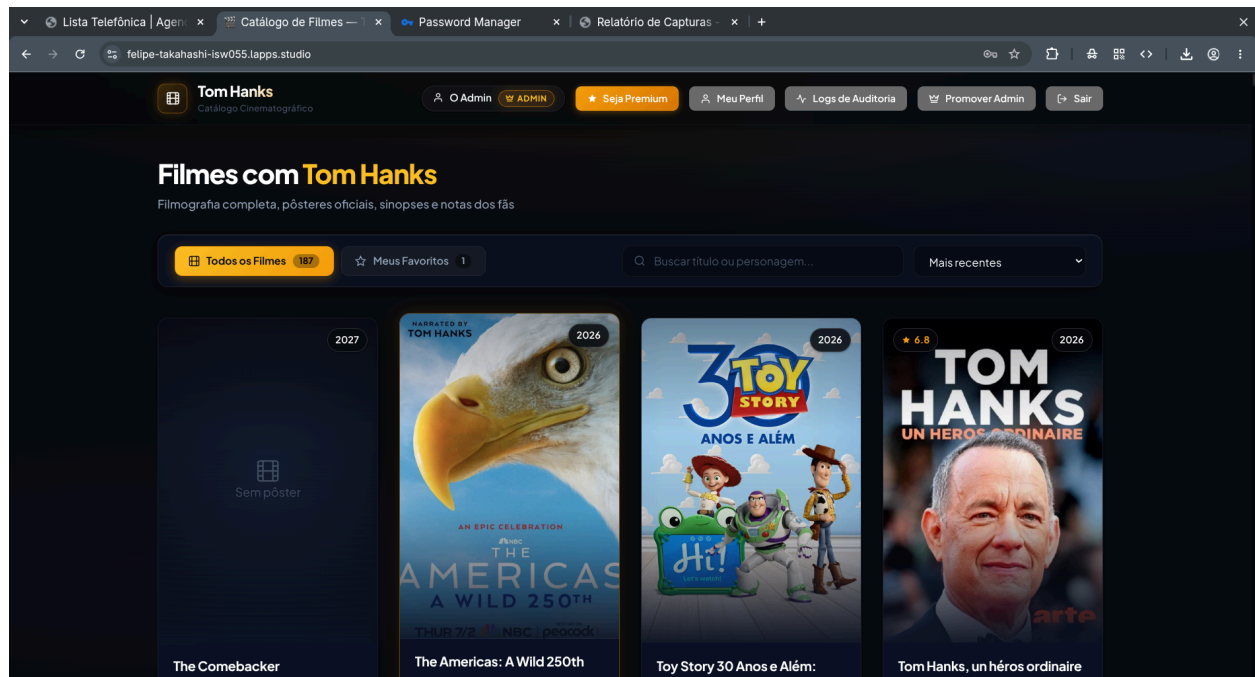


Figura 5: Atividade 2 — resultado funcional (Catálogo de Filmes com Tom Hanks em execução no navegador)

Dificuldades e como foram resolvidas. Durante o deploy no ambiente do Portainer, foram identificados conflitos gerados por nomes estáticos de containers e vinculação desnecessária de portas do MariaDB diretamente na interface de rede do host. O problema foi sanado com o commit 230de48, removendo o mapeamento de portas externas no banco de dados e padronizando variáveis de ambiente no Compose, viabilizando a inicialização limpa da stack na porta 3000.

4.3 Atividade 3 — Desacoplando o login — microserviço de autenticação

Descrição	Login, cadastro e esqueci-minha-senha em microserviço à parte na rede interna do Docker com tokens de 30 min.
Data planejada	28/08/2026
Data realizada	28/08/2026 13:56
Situação	entregue
Evidência	GitHub — commit 6d5adcb + docker-compose.yml + Issue #2 + prints do Mailtrap
Link	https://github.com/Felipe-Takayuki/atividade-filmes/issues/2

O que foi feito. Realizou-se a primeira grande refatoração arquitetural do sistema, desacoplando completamente as responsabilidades de controle de identidade do monólito e criando o `auth-service` como um microserviço independente em diretório e container próprio. O novo serviço assumiu o ciclo de vida completo de cadastro de usuários com hash criptográfico `bcrypt`, autenticação com geração de tokens JWT e o fluxo de recuperação de senha (“Esqueci minha senha”). Para isso, foram criados tokens aleatórios seguros de 32 bytes armazenados na tabela `reset_tokens` com validade estrita de 30 minutos e invalidação imediata após o primeiro uso (`usado = TRUE`). A infraestrutura foi orquestrada no Docker Compose com a rede `bridge app-network`: o container `auth-service` opera isolado com `expose: [ "4000" ]` (sem portas publicadas para o host), enquanto o `catalogo` atua como proxy reverso e gateway de entrada. O fluxo foi comprovado com envio real de e-mails via Mailtrap e registrado detalhadamente na Issue #2.

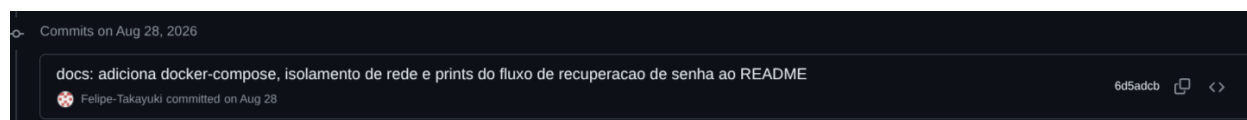


Figura 6: Atividade 3 — evidência da entrega (commit 6d5adcb e documentação da Issue 2 em 28/08/2026)

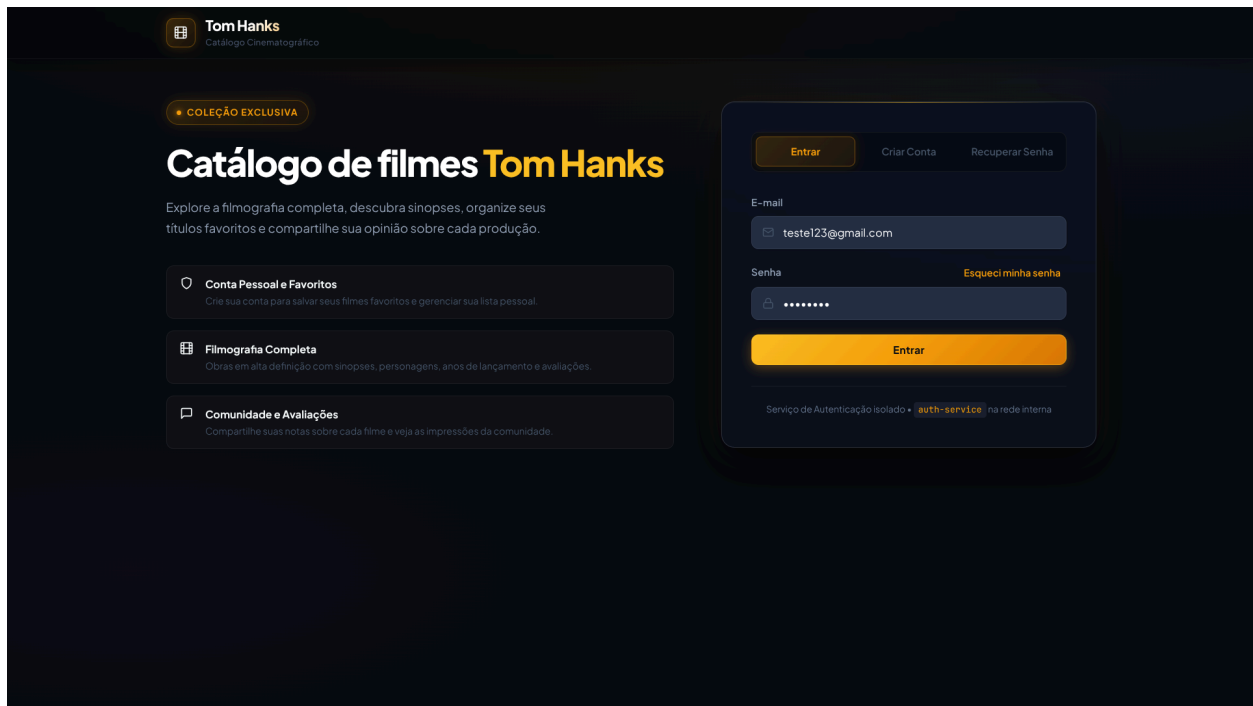


Figura 7: Atividade 3 — tela de login consumindo o microserviço auth-service isolado na rede interna

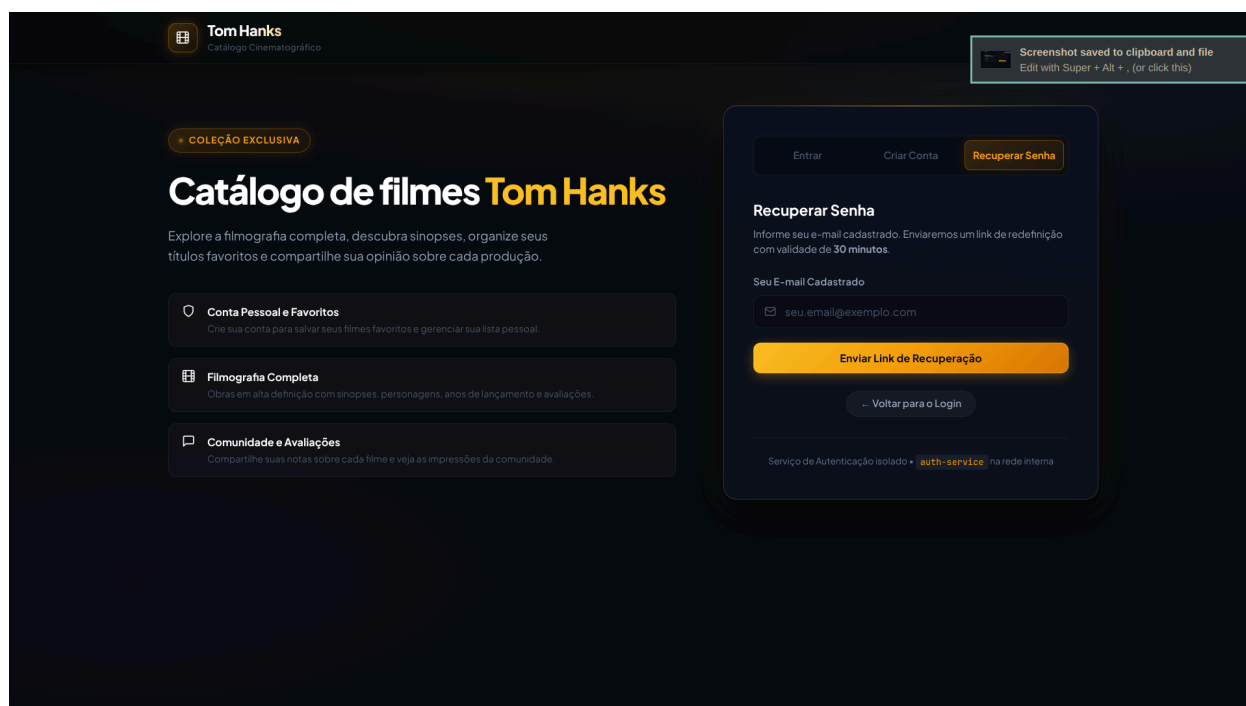


Figura 8: Atividade 3 — fluxo de recuperação de senha com formulário e link de redefinição de uso único (30 minutos)

Dificuldades e como foram resolvidas. O desafio consistiu em garantir que requisições originadas no navegador cliente conseguissem consumir as operações de autenticação sem expor o `auth-service` publicamente na internet. A solução foi implementar um cliente HTTP interno no backend do catálogo (`AUTH_SERVICE_URL=http://auth-service:4000`), encaminhando cabeçalhos de autorização e tratando erros de rede com respostas padronizadas, além de validar no frontend a rejeição de links expirados ou já consumidos.

4.4 Atividade 4 — Controle de acesso por papel — RBAC

Descrição	Autorização baseada em papéis com enforcement no backend (403 para usuário comum), moderação e promoção.
Data planejada	04/09/2026
Data realizada	04/09/2026 16:50
Situação	entregue
Evidência	GitHub — commit e5228cc + Issue #3 + moderação admin e bloqueio 403
Link	https://github.com/Felipe-Takayuki/atividade-filmes/issues/3

O que foi feito. Implementou-se um sistema completo de Controle de Acesso Baseado em Papéis (RBAC - **Role-Based Access Control**), estruturado em torno de dois níveis hierárquicos: **usuario** (papel padrão atribuído automaticamente aos novos cadastros, com permissão para visualizar o catálogo, gerenciar sua própria lista de favoritos e criar/excluir apenas seus próprios comentários) e **admin** (administrador do sistema com poderes de moderação global de comentários e concessão de privilégios a outros usuários). Foi adotado o padrão arquitetural de **Enforcement Centralizado** (Padrão A), no qual cada operação sensível consulta o papel em tempo real no banco/auth-service, garantindo revogação ou concessão imediata de permissões. O backend rejeita terminantemente requisições não autorizadas com código HTTP 403 Forbidden (FORBIDDEN_NOT_ADMIN). Na interface gráfica (React), foram introduzidos badges visuais (admin em dourado e usuario em azul), botões contextuais de moderação e o modal administrativo Promover Admin por E-mail, tudo documentado na Issue #3.

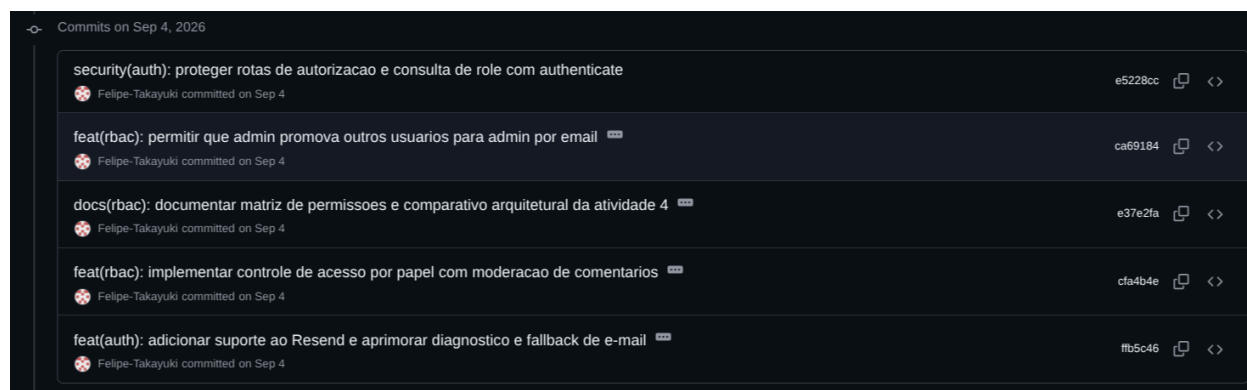


Figura 9: Atividade 4 — evidência da entrega (commit e5228cc e submissão da Issue 3 em 04/09/2026)

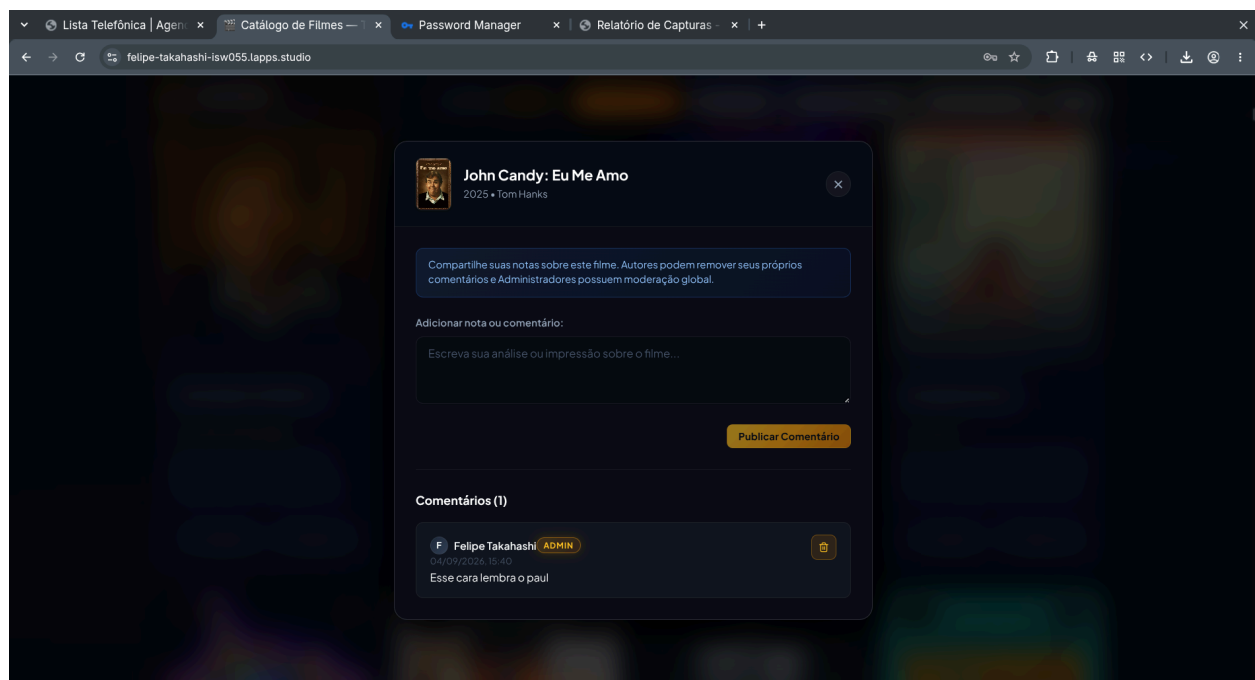


Figura 10: Atividade 4 — moderação de comentários com badge ADMIN e exclusão de análises de qualquer usuário

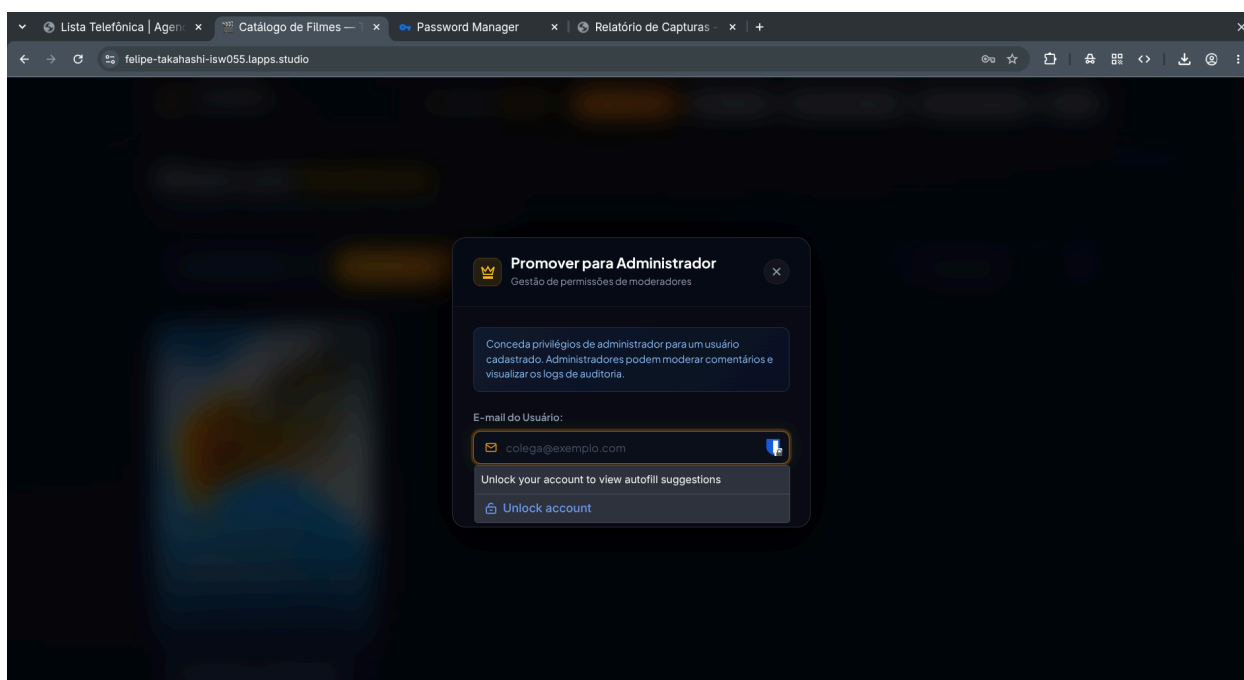


Figura 11: Atividade 4 — modal administrativo para promoção de usuários cadastrados ao papel de ADMIN

Dificuldades e como foram resolvidas. Um ponto crítico de segurança foi evitar vulnerabilidades de atribuição em massa (**mass assignment**), na qual requisições maliciosas enviadas via cURL ou Postman para o endpoint de cadastro tentassem injetar o atributo `role: "admin"`. Para eliminar esse risco, a rota `POST /api/auth/register` foi blindada no servidor, forçando irrevogavelmente o valor `role = 'usuario'` para todo novo registro e exigindo que qualquer elevação de cargo seja autenticada e disparada exclusivamente pela rota interna protegida de administração.

4.5 Atividade 5 — Logs e auditoria

Descrição	Novo microserviço log-service com Redis Streams (XADD/XREVRANGE) registrando logins, ações e tentativas 403.
Data planejada	25/09/2026
Data realizada	10/09/2026 15:48
Situação	entregue
Evidência	GitHub — commit b024bb0 + Issue #4 + log-service e Redis Streams
Link	https://github.com/Felipe-Takayuki/atividade-filmes/issues/4

O que foi feito. Criação e integração do terceiro microserviço da arquitetura: o log-service (porta interna :5000), desenvolvido em Node.js e acoplado a uma instância do banco NoSQL Redis 7 com persistência contínua AOF (**Append-Only File**). O serviço foi projetado para registrar de maneira centralizada e assíncrona todas as trilhas de auditoria da plataforma através de Redis Streams via comando nativo XADD sobre a chave audit:events. O sistema monitora ativamente operações como login, logout, adição/remoção de favoritos, criação/exclusão de comentários e, criticamente, incidentes de segurança com o evento acao_negada_403. Como bônus de rastreabilidade, cada registro captura o endereço IP de origem do cliente e dados contextuais da requisição. Para consulta, foi criada a rota administrativa GET /api/logs, restrita ao papel admin (com proteção RBAC 403), que lê a stream em ordem cronológica inversa usando XREVRANGE e alimenta um modal interativo na interface web. A atividade foi entregue e homologada na Issue #4 com 15 dias de antecedência.

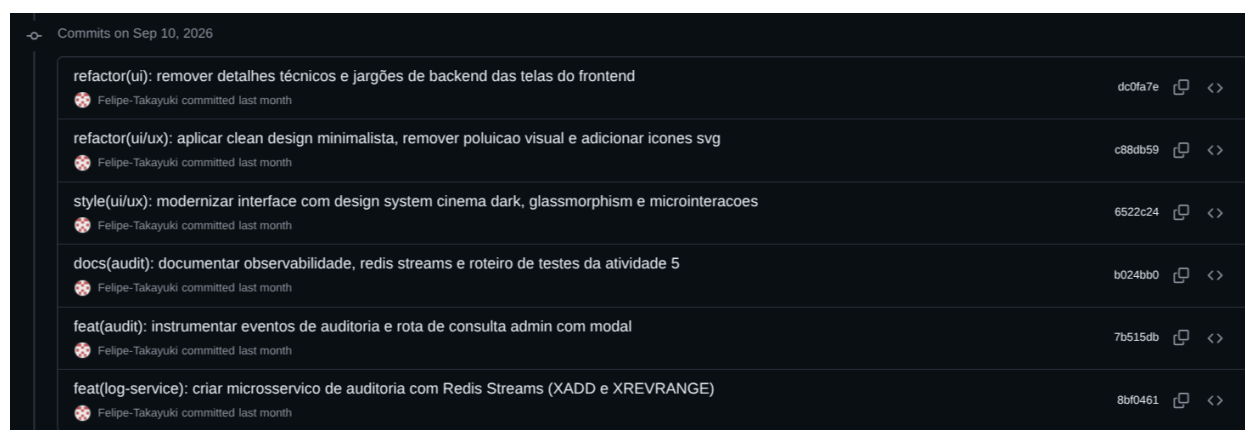


Figura 12: Atividade 5 — evidência da entrega (commit b024bb0 e Issue 4 entregue com 15 dias de antecedência)

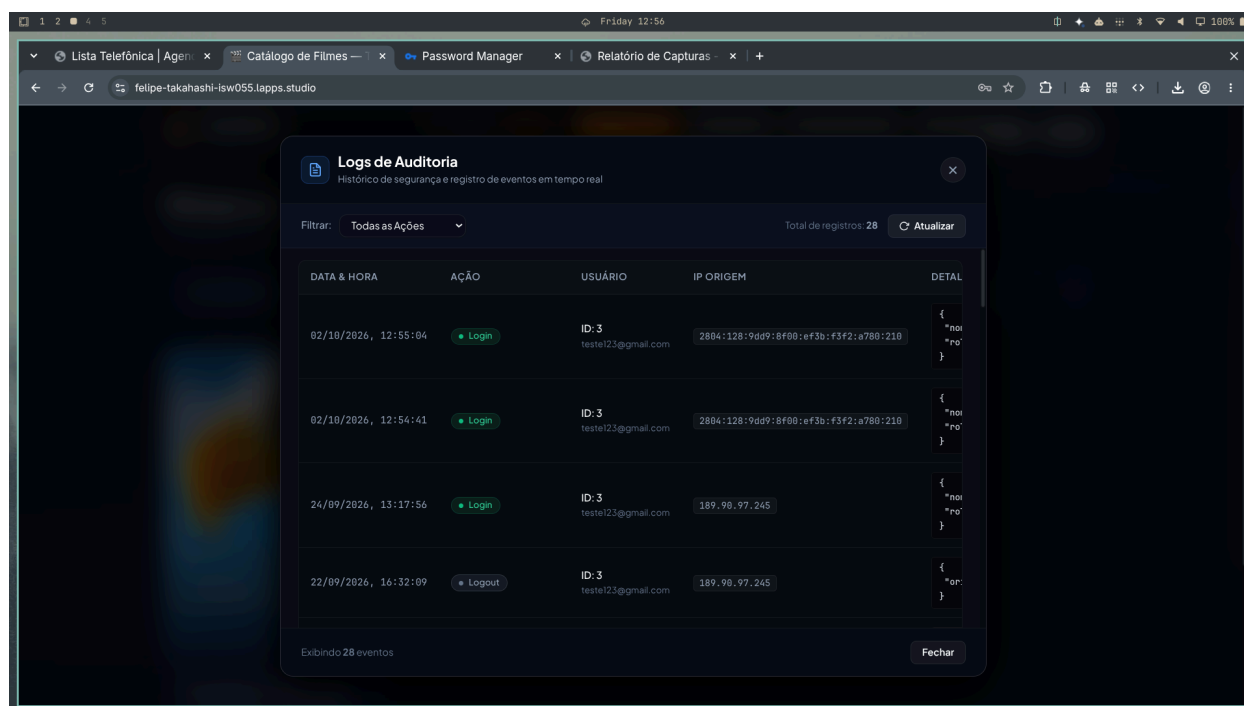


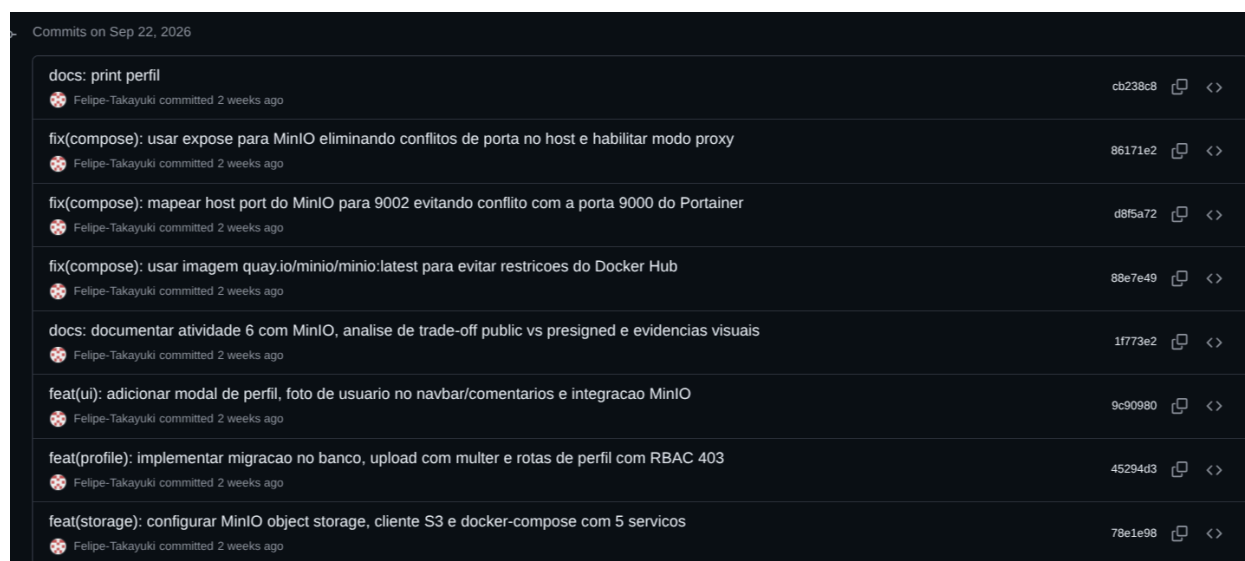
Figura 13: Atividade 5 — resultado funcional (painel de auditoria do log-service com Redis Streams e IPs de origem)

Dificuldades e como foram resolvidas. Havia a preocupação de que lentidões na escrita de logs de auditoria pudessem degradar o tempo de resposta percebido pelo usuário final no catálogo. Esse problema foi solucionado desacoplando os disparos de log no backend através de chamadas HTTP não-bloqueantes (**fire-and-forget**), protegidas com cláusulas catch e timeouts curtos (2 segundos), garantindo que a aplicação permaneça perfeitamente funcional mesmo caso o serviço de logs sofra manutenções.

4.6 Atividade 6 — Upload e perfil de usuário

Descrição	Página de perfil com avatar no MinIO Object Storage (S3), metadados no MariaDB e proteção estrita RBAC 403.
Data planejada	02/10/2026
Data realizada	22/09/2026 15:38
Situação	entregue
Evidência	GitHub — commit cb238c8 + Issue #5 + MinIO e perfil com foto
Link	https://github.com/Felipe-Takayuki/atividade-filmes/issues/5

O que foi feito. Implementação da arquitetura de Armazenamento de Objetos (**Object Storage**) compatível com o protocolo AWS S3, introduzindo o container MinIO com volume dedicado minio-data e bucket catalogo-perfil. Foi desenvolvida a funcionalidade completa de perfil do usuário: upload de avatares com validação rigorosa de mime-type (apenas imagens) e limite máximo de 5MB utilizando a biblioteca Multer; envio do binário diretamente para o MinIO gerando chaves determinísticas de objeto (avatars/user-ID-timestamp.png); e persistência relacional leve no MariaDB, que armazena exclusivamente a referência da foto (foto_key), o nome e a biografia do usuário. Para a segurança, aplicou-se o princípio da confiança zero: o backend confere o token JWT e bloqueia qualquer tentativa de um usuário consultar e-mails ou alterar o perfil de terceiros com código HTTP 403 Forbidden (FORBIDDEN_PROFILE_EDIT). No README e na Issue #5 foi documentada uma profunda análise de trade-offs entre buckets de leitura pública versus URLs pré-assinadas com HMAC, acompanhada de 10 testes automatizados aprovados e entrega realizada com 10 dias de antecedência.



Commits on Sep 22, 2026		
docs: print perfil	Felipe-Takayuki committed 2 weeks ago	cb238c8
fix(compose): usar expose para MinIO eliminando conflitos de porta no host e habilitar modo proxy	Felipe-Takayuki committed 2 weeks ago	86171e2
fix(compose): mapear host port do MinIO para 9002 evitando conflito com a porta 9000 do Portainer	Felipe-Takayuki committed 2 weeks ago	d8f5a72
fix(compose): usar imagem quay.io/minio/minio:latest para evitar restricoes do Docker Hub	Felipe-Takayuki committed 2 weeks ago	88e7e49
docs: documentar atividade 6 com MinIO, analise de trade-off public vs presigned e evidencias visuais	Felipe-Takayuki committed 2 weeks ago	1f773e2
feat(ui): adicionar modal de perfil, foto de usuario no navbar/comentarios e integracao MinIO	Felipe-Takayuki committed 2 weeks ago	9c90980
feat(profile): implementar migracao no banco, upload com multer e rotas de perfil com RBAC 403	Felipe-Takayuki committed 2 weeks ago	45294d3
feat(storage): configurar MinIO object storage, cliente S3 e docker-compose com 5 servicos	Felipe-Takayuki committed 2 weeks ago	78e1e98

Figura 14: Atividade 6 — evidência da entrega (commit cb238c8 e submissão da Issue 5 com 10 dias de antecedência)

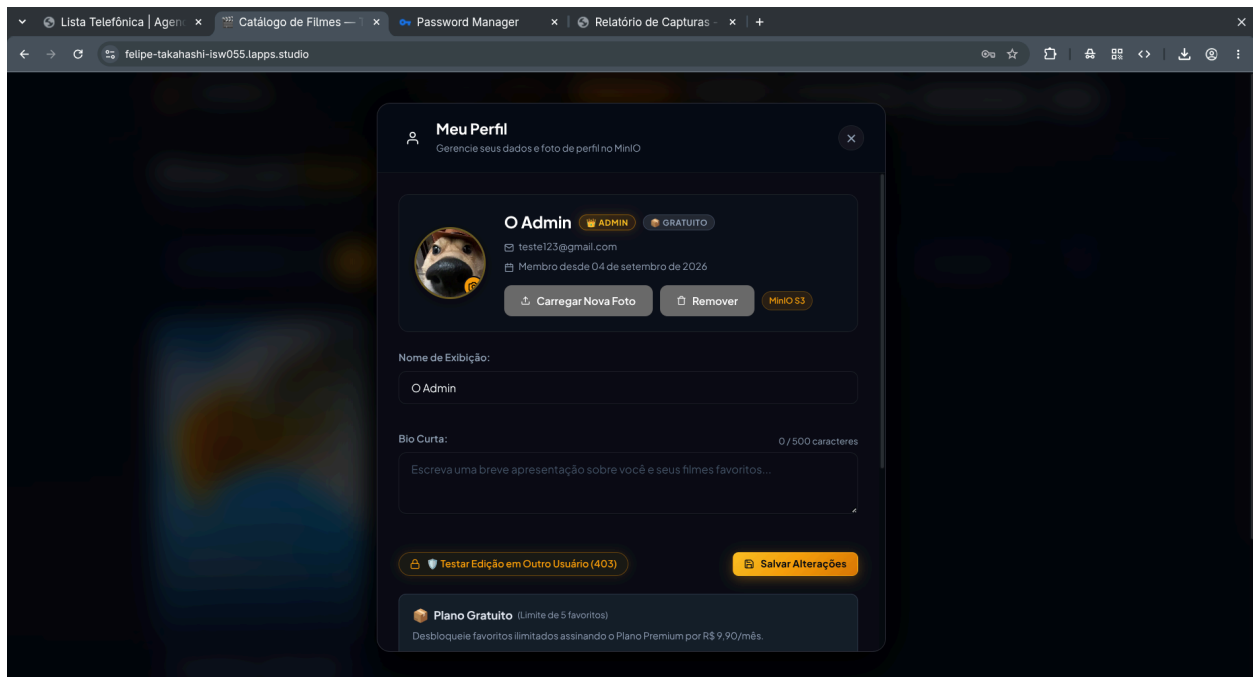


Figura 15: Atividade 6 — resultado funcional (perfil com foto no MinIO, bio, dados e teste de proteção 403)

Dificuldades e como foram resolvidas. Em ambientes de hospedagem compartilhados, a porta padrão :9000 do MinIO conflitou diretamente com a porta da API do Portainer pré-instalado na máquina. A dificuldade foi resolvida no Docker Compose eliminando a publicação de portas externas no host (`expose: [ "9000", "9001" ]`) e desenvolvendo uma rota interna de proxy seguro no backend do catálogo (`GET /api/profile/avatar/:fotoKey`), permitindo servir os avatares aos navegadores clientes de forma universal sem qualquer interferência de rede.

5 Atividades extras (opcional)

5.1 Atividade E3 — Observabilidade — health checks e métricas

Descrição	Endpoints de saúde e integridade em cascata (/api/health) e dependência com service_healthy no Docker Compose.
Data planejada	sem prazo
Data realizada	10/09/2026 15:48
Situação	entregue
Evidência	GitHub — commit 8bf0461 + endpoints /health em cascata + README oficial
Link	https://github.com/Felipe-Takayuki/atividade-filmes/commit/8bf0461

O que foi feito. Implementação de uma esteira robusta de verificação de integridade (**health checks**) distribuída entre os microsserviços do sistema. Foi criado o endpoint mestre GET /api/health no catálogo, complementado pelos endpoints /health no auth-service e no log-service. A rota inspeciona ativamente o status da conexão com a base relacional MariaDB executando a query `SELECT 1`, afere a latência e conectividade HTTP interna com o serviço de autenticação, valida a disponibilidade do serviço de logs e verifica a persistência dos streams no Redis. No arquivo `docker-compose.yml`, configurou-se uma checagem periódica nativa para o Redis (`redis-cli ping` a cada 5 segundos), instruindo o `log-service` a depender explicitamente da condição `condition: service_healthy`, garantindo que o cluster inicie em ordem determinística sem `race conditions`.

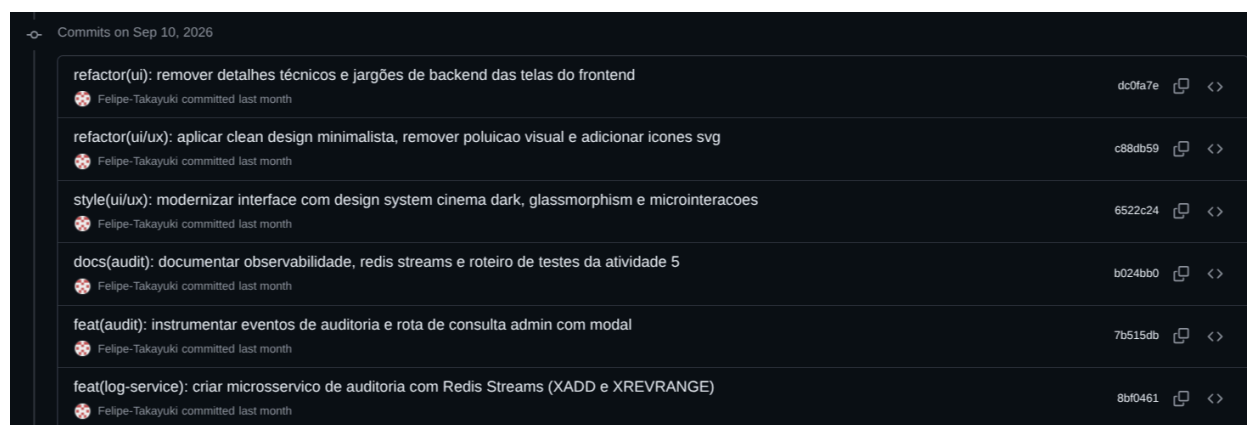
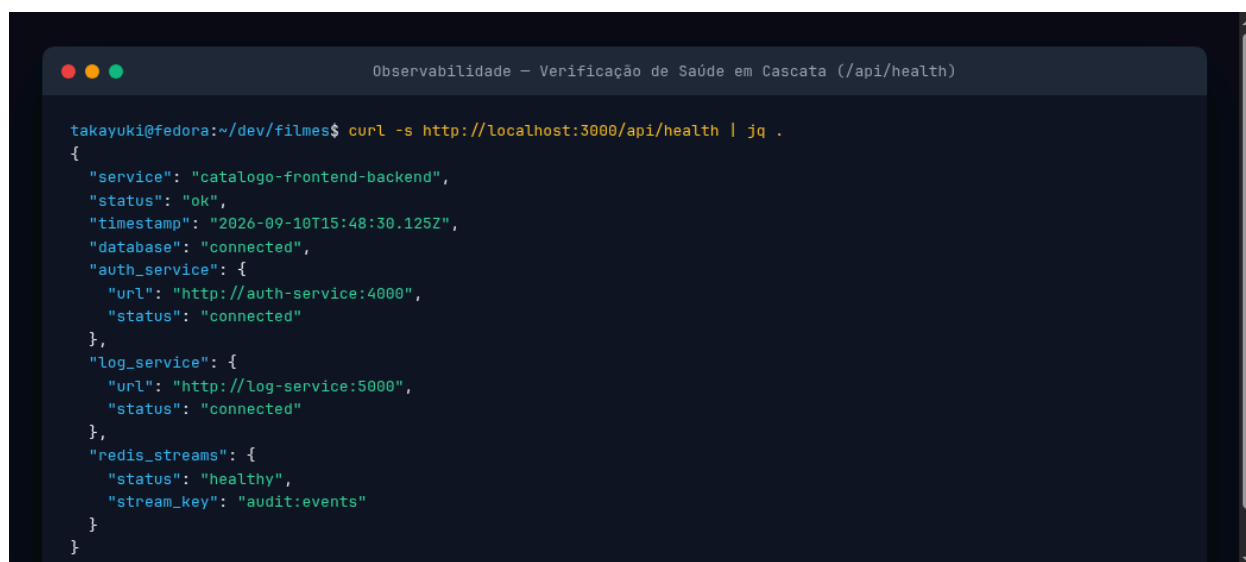


Figura 16: Atividade E3 — evidência da entrega (commit 8bf0461 instrumentando a observabilidade em cascata)



```
Observabilidade — Verificação de Saúde em Cascata (/api/health)

takayuki@fedora:~/dev/filmes$ curl -s http://localhost:3000/api/health | jq .
{
  "service": "catalogo-frontend-backend",
  "status": "ok",
  "timestamp": "2026-09-10T15:48:30.125Z",
  "database": "connected",
  "auth_service": {
    "url": "http://auth-service:4000",
    "status": "connected"
  },
  "log_service": {
    "url": "http://log-service:5000",
    "status": "connected"
  },
  "redis_streams": {
    "status": "healthy",
    "stream_key": "audit:events"
  }
}
```

Figura 17: Atividade E3 — resultado (resposta JSON estruturada do health check /api/health)

Dificuldades e como foram resolvidas. O desafio foi estruturar a resposta do health check de modo que uma falha transitória em um serviço secundário não derrubasse a resposta geral do endpoint principal. A solução foi encapsular as chamadas internas com `AbortSignal.timeout(2000)` e categorizar o status dos serviços em três níveis (`connected`, `degraded`, `offline`), permitindo que a equipe de operações identifique imediatamente qual elo da infraestrutura demanda atenção.

6 Considerações finais

A trajetória percorrida ao longo do primeiro bimestre de Introdução à Computação em Nuvem consolidou um salto qualitativo fundamental na maturidade técnica do desenvolvimento de software em arquiteturas distribuídas. Mais do que assimilar conceitos teóricos sobre containers e microsserviços, a oportunidade de construir passo a passo o Catálogo de Filmes exigiu o enfrentamento diário de problemas reais de engenharia: particionamento de responsabilidades entre serviços, comunicação segura em redes isoladas do Docker, tratamento de latências e falhas parciais, além da clara distinção entre persistência relacional transacional (MariaDB), bancos em memória para auditoria (Redis Streams) e armazenamento de grandes objetos não estruturados (MinIO S3).

O aspecto de maior complexidade técnica enfrentado no período residiu na implementação do Controle de Acesso Baseado em Papéis (RBAC) combinado com o isolamento de rede e a compatibilidade com a infraestrutura do Portainer. Foi necessário auditar minuciosamente cada rota da API para impedir brechas como **mass assignment** e vazamento de informações entre usuários, garantindo que o backend fizesse o enforcement absoluto das regras de negócio através de respostas HTTP 403 Forbidden. Caso tivesse que reiniciar o projeto, teria adotado desde o primeiro momento a rota proxy para o MinIO e um gateway padronizado para as chamadas internas, evitando os ajustes retroativos de portas que foram necessários durante a Atividade 6.

Para o segundo bimestre, pretendo utilizar a sólida arquitetura estabelecida como alicerce para a evolução da plataforma, explorando automações com esteiras de Integração e Entrega Contínuas (CI/CD) no GitHub Actions, orquestração avançada e o fechamento do ciclo de produto com a implementação de planos de assinatura e monetização (SaaS com Stripe), assegurando que o sistema continue escalável, resiliente e altamente auditável.

7 Declaração de autoria

Declaro que este relatório foi elaborado por mim, individualmente, e que as evidências apresentadas correspondem a entregas de minha autoria, verificáveis nos links informados. Nas atividades realizadas em grupo ou nivelamentos em sala, o conteúdo aqui descrito refere-se integralmente à minha participação e desenvolvimento técnico.

Felipe Takayuki Tanaka Takahashi

Pompeia, 07/10/2026